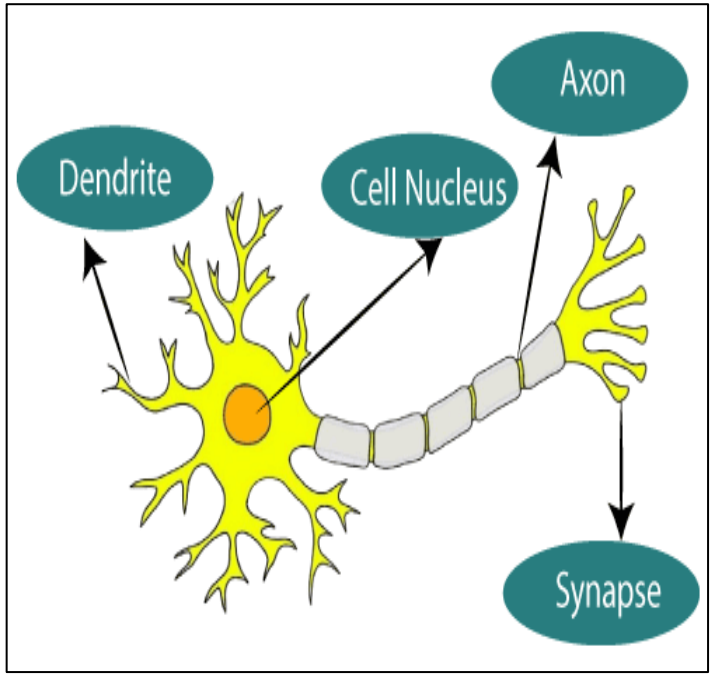
Machine Learning with Python
BCA-311
Unit-3 Notes

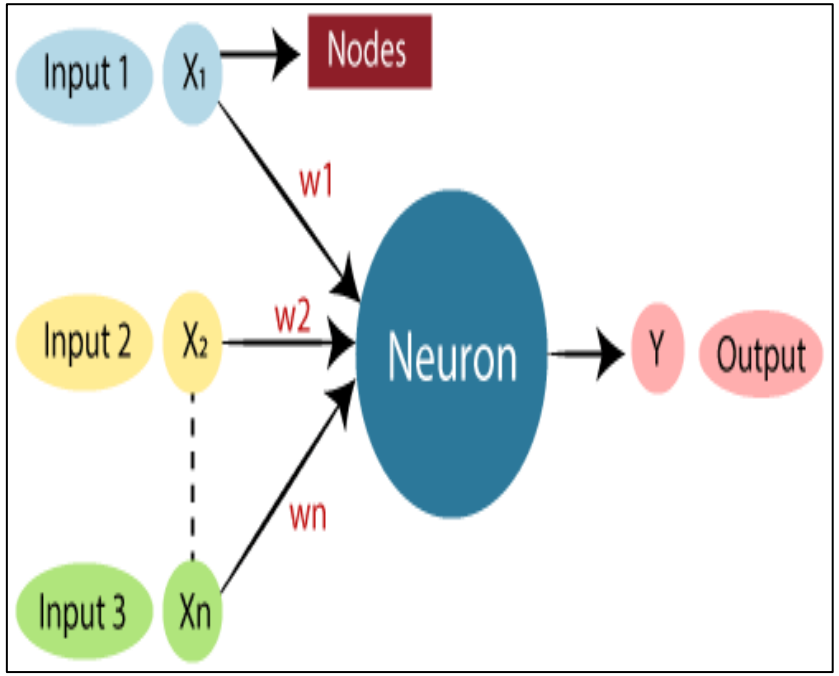
Syllabus covered

- Artificial Neural Networks, HebbNet, Perceptron, Adaline, Multilayer Neural Network, Architecture, Activation Functions, Loss Function, Hyper parameters, Gradient Descent, Backpropagation, Variants of Backpropagation, Avoiding overfitting through Regularization, Applications of Neural Networks.

Artificial Neural Networks

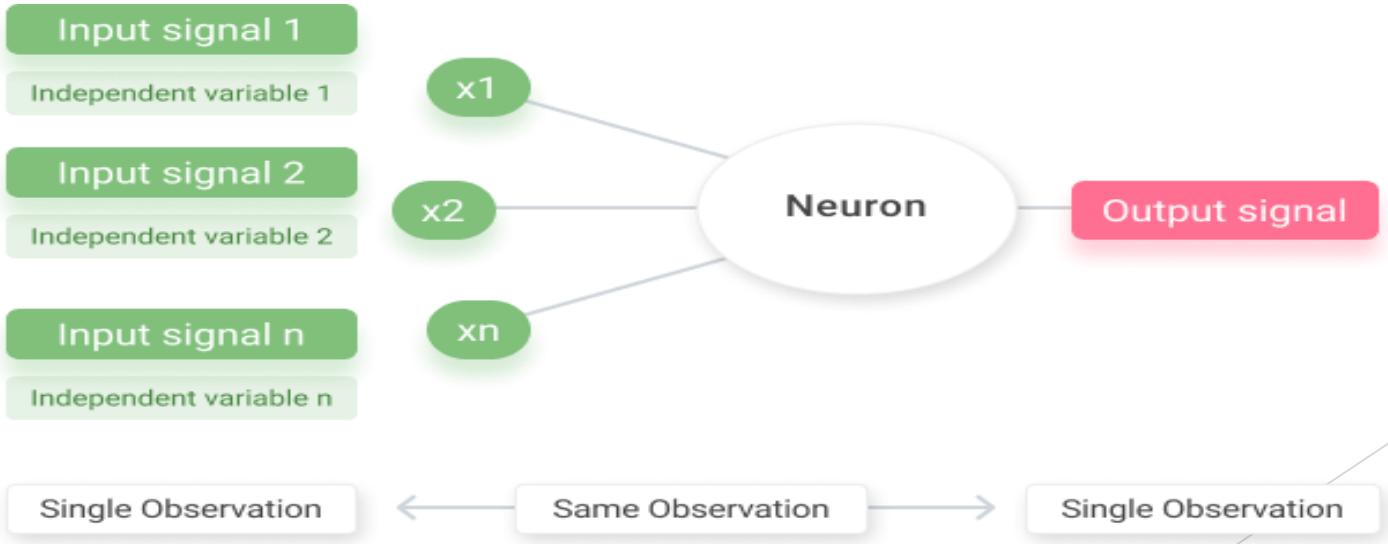
- The term "Artificial Neural Network" is derived from Biological neural networks that develop the structure of a human brain. Similar to the human brain that has neurons interconnected to one another, artificial neural networks also have neurons that are interconnected to one another in various layers of the networks. These neurons are known as **nodes.**





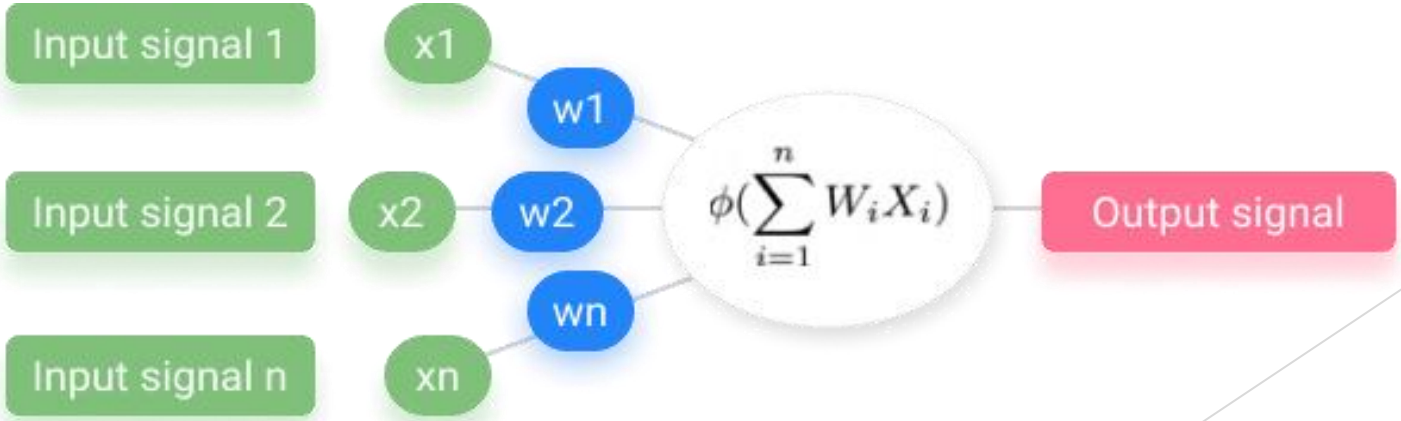
Biological Neural Network	Artificial Neural Network
Dendrites	Inputs
Cell nucleus	Nodes
Synapse	Weights
Axon	Output

- Neurons have branches coming out of them from both ends, called dendrites. One neuron can't do much, but when thousands of neurons connect and work together, they are powerful and can process complex actions and concepts. A computer node works in the same way a human neuron does and replicates real neurons.
- For our digital neuron, independent values (input signals) get passed through the "neuron" in order to generate a dependent value (output signal). These independent variables in one layer are just a row of data for one single observation.
- For example, in the context of a neural network problem, one input layer would signify one variable – maybe the age or gender (independent variable) of a person whose identity (dependent) we are trying to figure out.



Weights and Activation

- The synapses are those lines connecting the “neuron” to the input signals. Weights are assigned to all of them. Weights are crucial to artificial neural networks because they let the networks “learn.” The weights decide which input signals are not important – which ones get passed along and which don’t.
- it takes the weighted sum of all the input values. It then applies an activation function. Activation functions are just functions applied to the weighted sum. Depending on the outcome of the applied function, the neuron will either pass on a signal or won’t pass it on.
- Most machine learning algorithms can be done in this type of form, with an array of input signals going through an activation function (which can be anything: logistic regression, polynomial regression, etc.), and an output signal at the end.



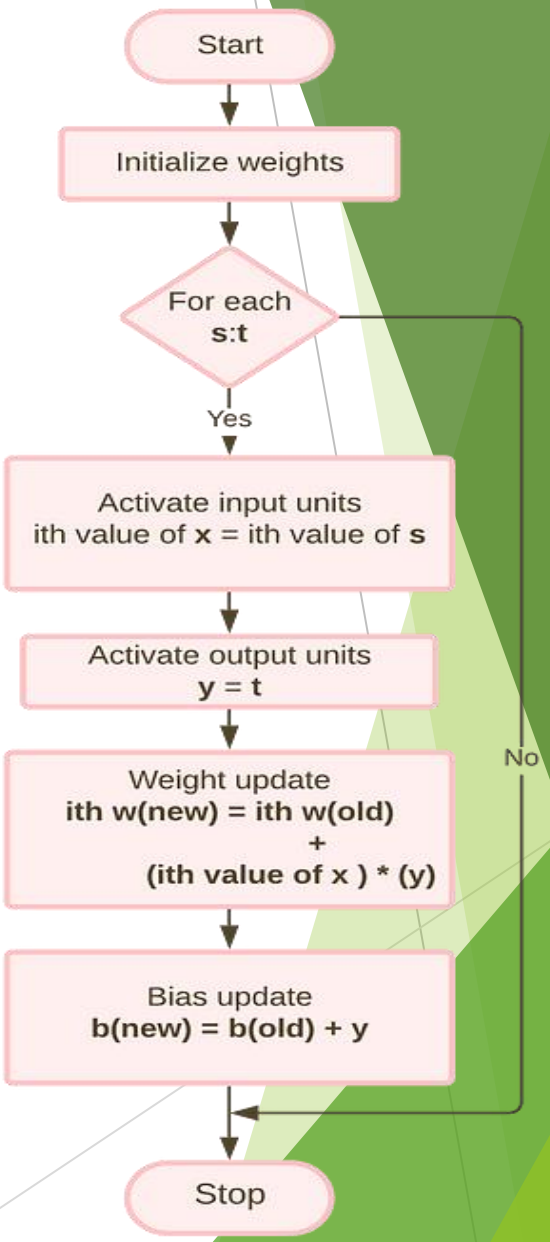
HebbNet

- Learning rule is a method or a mathematical logic. It helps a Neural Network to learn from the existing conditions and improve its performance. Thus learning rules updates the weights and bias levels of a network when a network simulates in a specific data environment. Applying learning rule is an iterative process. It helps a neural network to learn from the existing conditions and improve its performance.
- HebbNet is a neural network model that follows Hebbian learning rule.
- – It identifies, how to modify the weights of nodes of a network.
- The Hebbian rule was the first learning rule. In 1949 **Donald Hebb** developed it as learning algorithm of the unsupervised neural network.
- We can use it to identify how to improve the weights of nodes of a network.
- It is used for pattern classification. It is a single layer neural network, i.e. it has one input layer and one output layer. The input layer can have many units, say n. The output layer only has one unit. Hebbian rule works by updating the weights between neurons in the neural network for each training sample.

- The Hebb learning rule assumes that:
- When an axon of cell A is near enough to excite cell B and repeatedly or permanently takes place in firing it, some growth process or metabolic changes takes place in one or both the cells such that A's efficiency, as one of the cells firing B, is increased."
- This means learning is performed by the changes in the synaptic gap or in terms of NN by updating the weights.
- In this, if 2 interconnected neurons are ON simultaneously then the weight associated with these neurons can be increased by the modification made in their synaptic gaps(strength). The weight update in the Hebb rule is given by;

$$w_i(new) = w_i(old) + x_i y$$

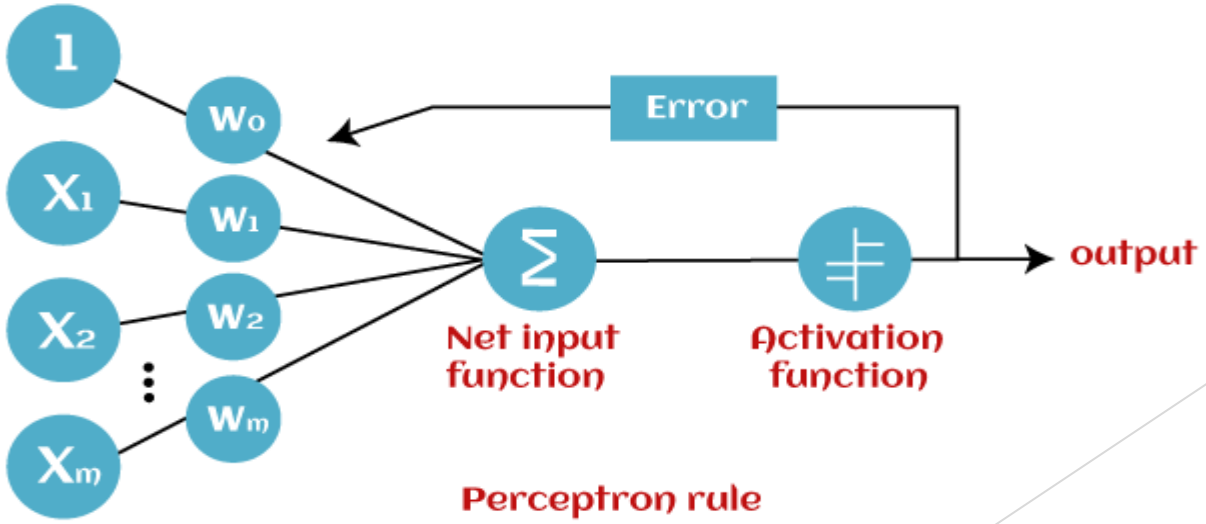
$$b(new) = b(old) + y$$



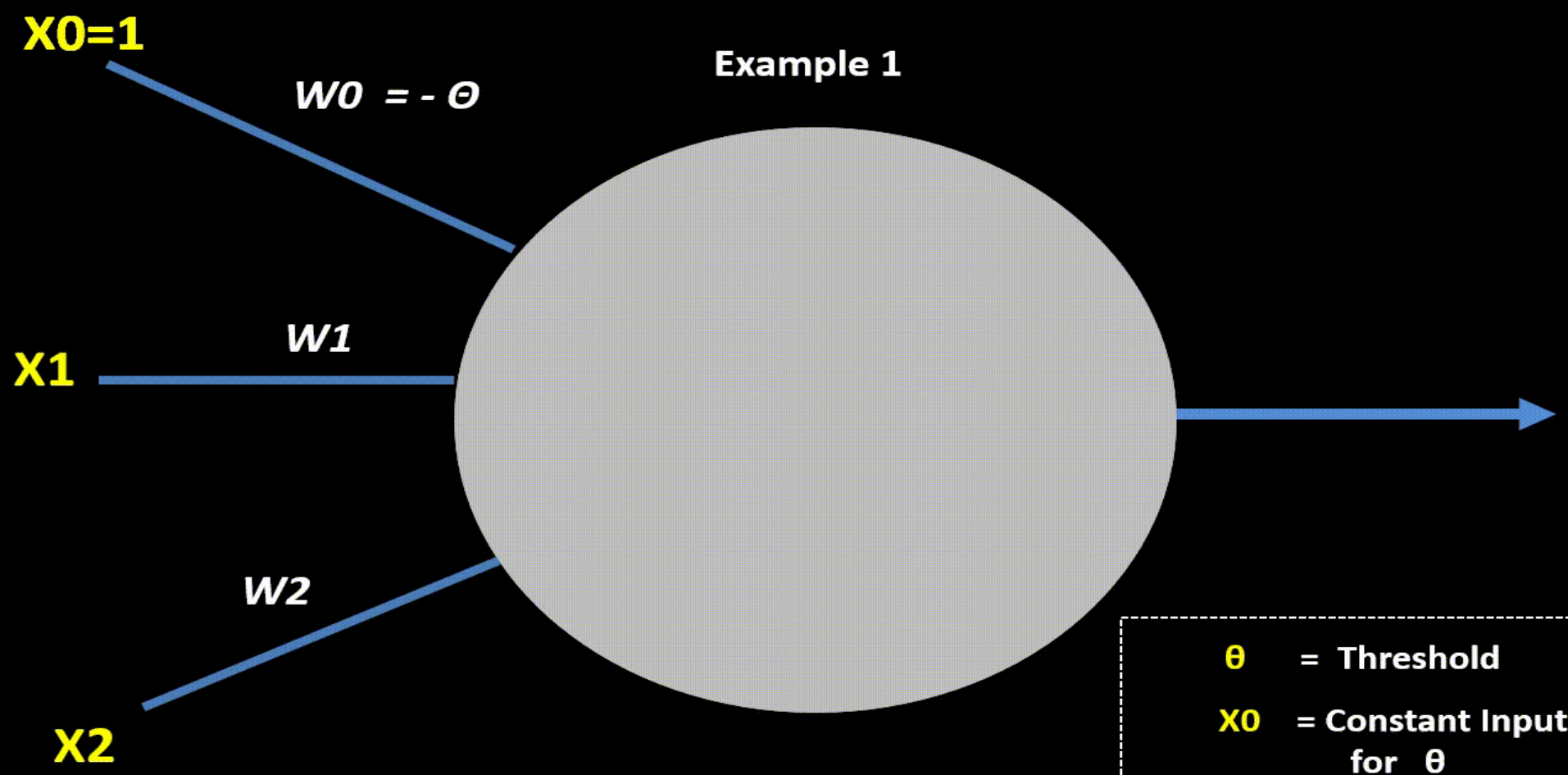
[Hebbian Learning Rule with Implementation of AND Gate - GeeksforGeeks]

Perceptron

- Perceptron is a building block of an Artificial Neural Network. Initially, in the mid of 19th century, **Mr. Frank Rosenblatt** invented the Perceptron for performing certain calculations to detect input data capabilities or business intelligence.
- Perceptron is a linear Machine Learning algorithm used for supervised learning for various **binary classifiers. "Single-layer perceptron can learn only linearly separable patterns."**
- Perceptron is considered as **a single-layer neural network** that consists of four main parameters named input values (Input nodes), weights and Bias, net sum, and an activation function. The perceptron model begins with the multiplication of all input values and their weights, then adds these values together to create the weighted sum. Then this weighted sum is applied to the activation function 'f' to obtain the desired output. This **activation function** is also known as **the step function** and is represented by **'f'**.
- The output is (0,1).



Perceptron Neuron (1957)



θ = Threshold

X0 = Constant Input 1 for θ

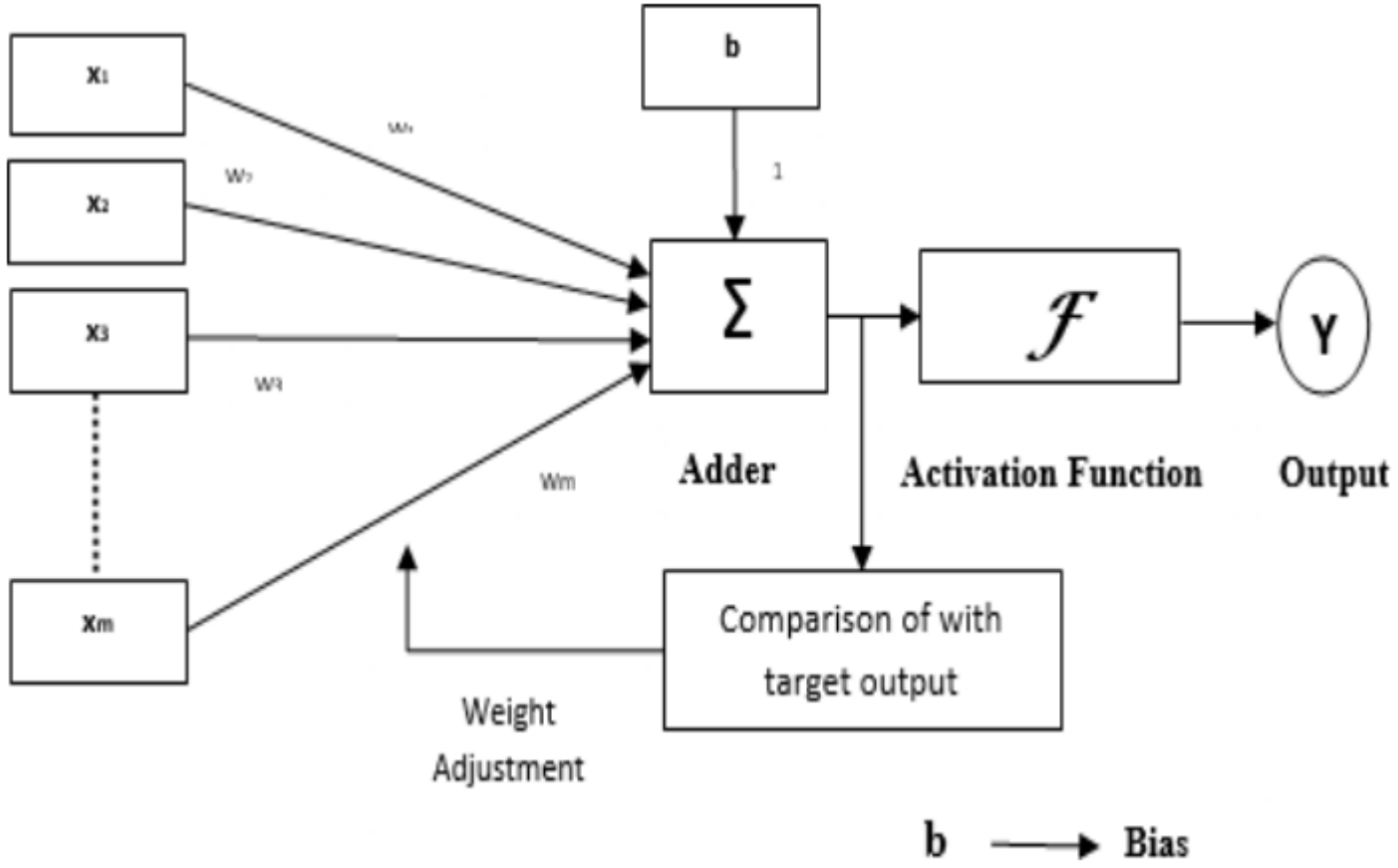
X1,X2 = Attribute Inputs

W1,W2 = Weights of Inputs X1,X2

Adaline

- ADALINE (Adaptive Linear Neuron or later Adaptive Linear Element) is an early single-layer artificial neural network and the name of the physical device that implemented this network.
- It was developed by Professor Bernard Widrow and his graduate student Ted Hoff at Stanford University in 1960. It is based on the McCulloch–Pitts neuron. It consists of a weight, a bias and a summation function.
- **The difference between Adaline and the standard (McCulloch-Pitts) perceptron is that in the learning phase, the weights are adjusted according to the weighted sum of the inputs (the net).**
- In the standard perceptron, the net is passed to the activation (transfer) function and the function's output is used for adjusting the weights. Some important points about Adaline are as follows:
- ▫ It uses bipolar activation function.
- ▫ It uses delta rule for training to minimize the Mean-Squared Error (MSE) between the actual output and the desired/target output.
- ▫ The weights and the bias are adjustable.

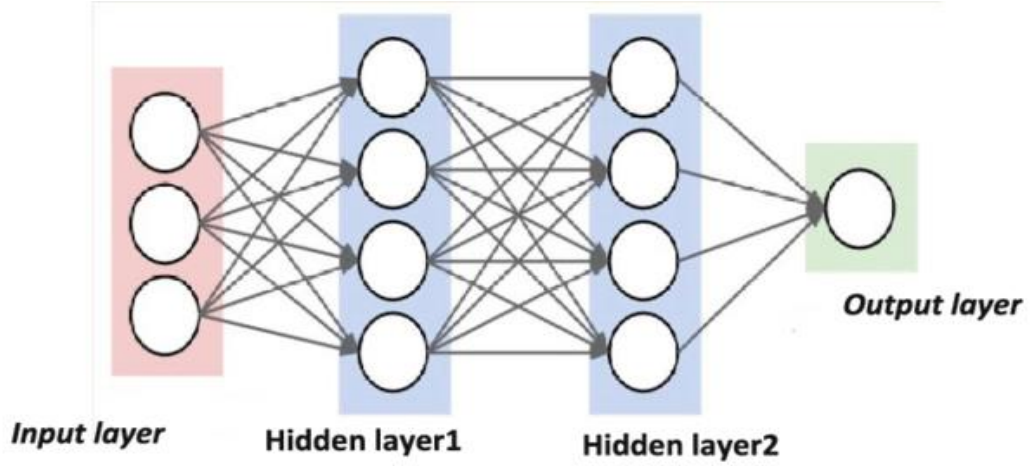
- Architecture of ADALINE network: The basic structure of Adaline is similar to perceptron having an extra feedback loop with the help of which the actual output is compared with the desired/target output. After comparison on the basis of training algorithm, the weights and bias will be updated.



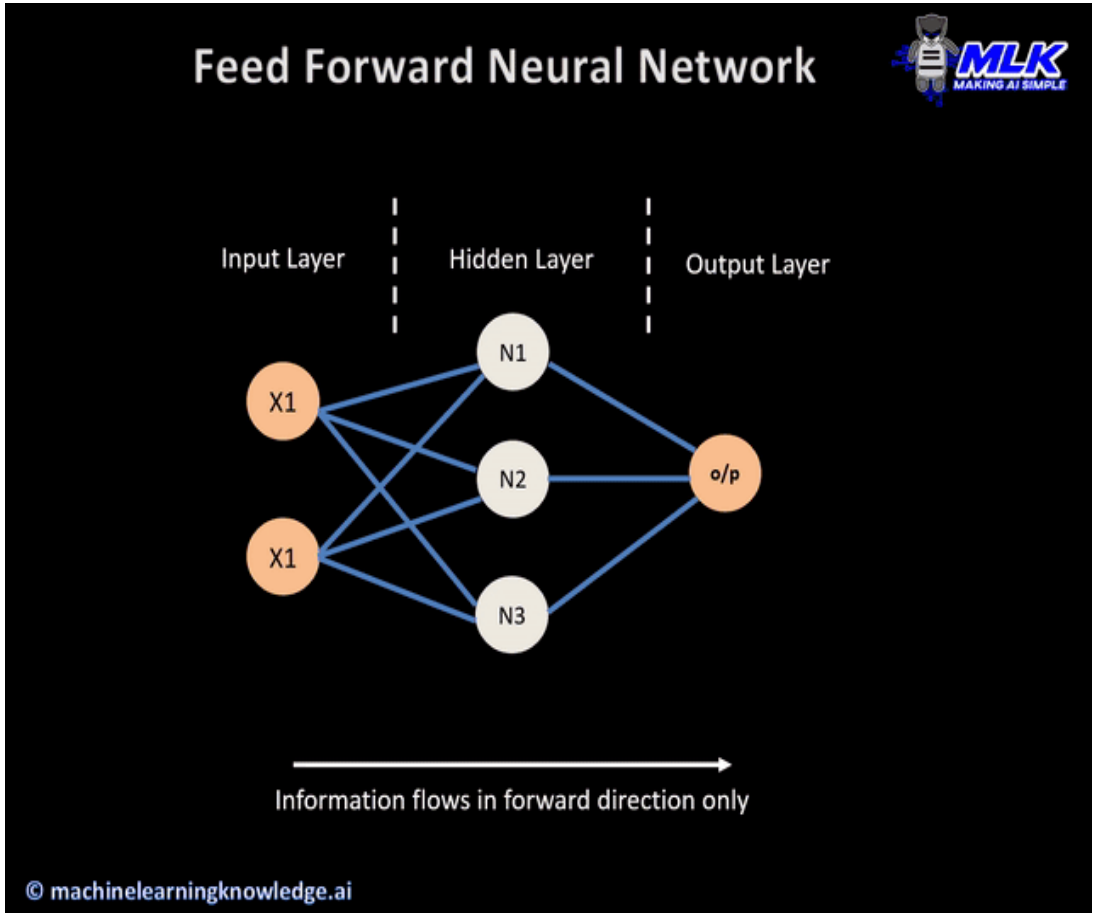
Multilayer Neural Network

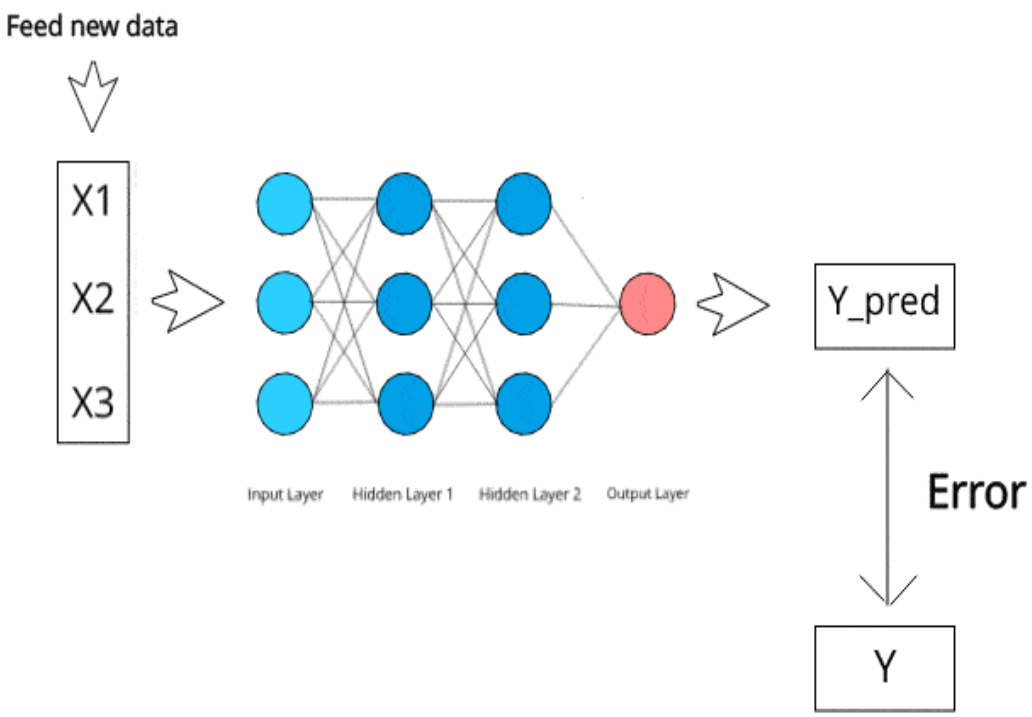
- **Multilayer Perceptron** also known as **Artificial Neural Networks** consists of more than one perceptron which is grouped together to form a multiple layer neural network.
- A multi-layered perceptron model can be considered as multiple artificial neural networks having various layers in which activation function does not remain linear, similar to a single layer perceptron model. Instead of linear, activation function can be executed as sigmoid, TanH, ReLU, etc., for deployment.
- The multi-layer perceptron model is also known as the Backpropagation algorithm, which executes in two stages as follows:
- **Forward Stage**: Activation functions start from the input layer in the forward stage and terminate on the output layer.
- **Backward Stage**: In the backward stage, weight and bias values are modified as per the model's requirement. In this stage, the error between actual output and demanded originated backward on the output layer and ended on the input layer.

- **Advantages of Multi-Layer Perceptron:**
- A multi-layered perceptron model can be used to solve complex non-linear problems.
- It works well with both small and large input data.
- It helps us to obtain quick predictions after the training.
- It helps to obtain the same accuracy ratio with large as well as small data.
- **Disadvantages of Multi-Layer Perceptron:**
- In Multi-layer perceptron, computations are difficult and time-consuming.
- In multi-layer Perceptron, it is difficult to predict how much the dependent variable affects each independent variable.
- The model functioning depends on the quality of the training.



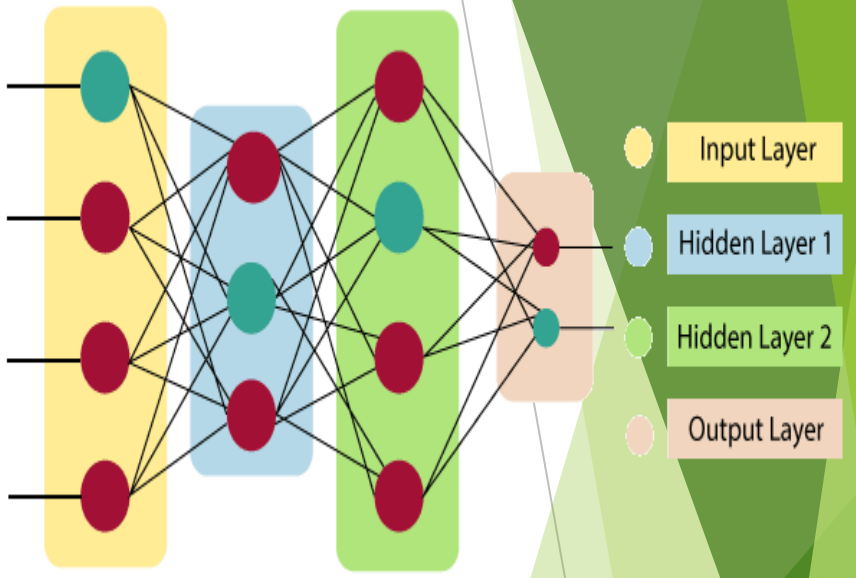
Working of Feed Forward and Back Propogation





Architecture

- Artificial Neural Network primarily consists of three layers:
- **Input Layer :** This layer accepts input features. It provides information from the outside world to the network, no computation is performed at this layer, nodes here just pass on the information(features) to the hidden layer.
- **Hidden Layer:** The hidden layer presents in-between input and output layers. It performs all the calculations to find hidden features and patterns.
- **Output Layer:** The input goes through a series of transformations using the hidden layer, which finally results in output that is conveyed using this layer.
- The artificial neural network takes input and computes the weighted sum of the inputs and includes a bias. This computation is represented in the form of a transfer function.
- The weighted total is passed as an input to an activation function to produce the output. Activation functions choose whether a node should fire or not. Only those who are fired make it to the output layer. There are distinctive activation functions available that can be applied upon the sort of task we are performing.



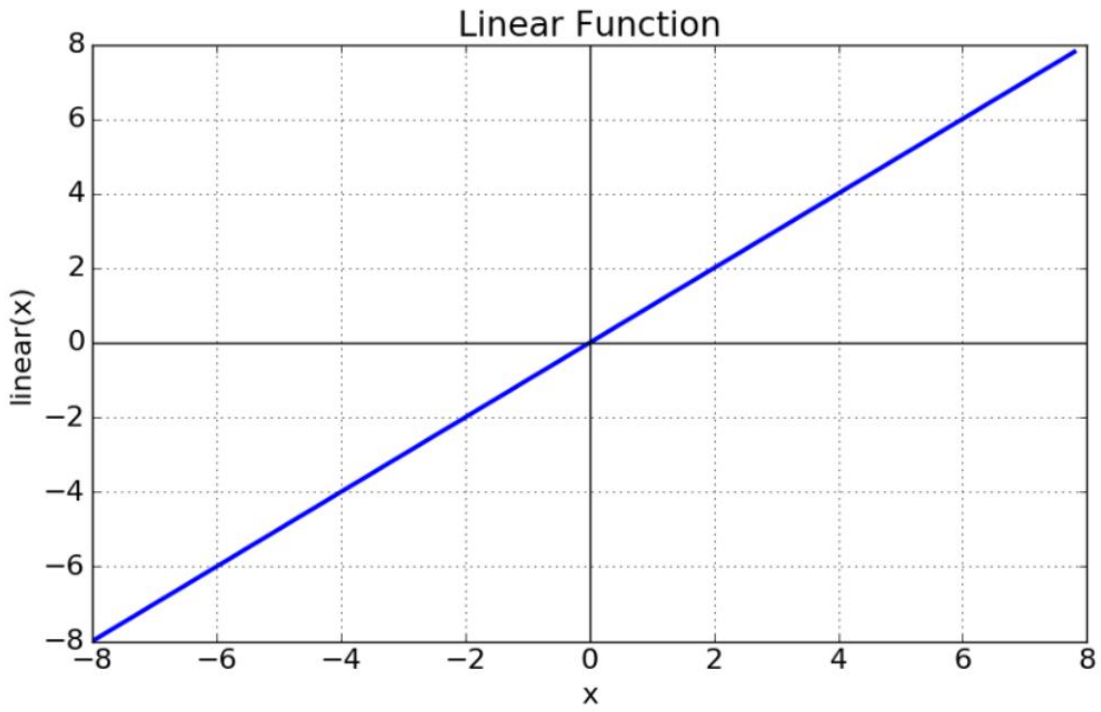
$$\sum_{i=1}^{n} Wi * Xi + b$$

Activation Functions

- In artificial neural networks, an activation function is one that outputs a smaller value for tiny inputs and a higher value if its inputs are greater than a threshold. An activation function "fires" if the inputs are big enough; otherwise, nothing happens. An activation function, then, is a gate that verifies how an incoming value is higher than a threshold value.
- It is also known as Transfer Function.
- The two main categories of activation functions are:
- Linear Activation Function
- Non-linear Activation Functions

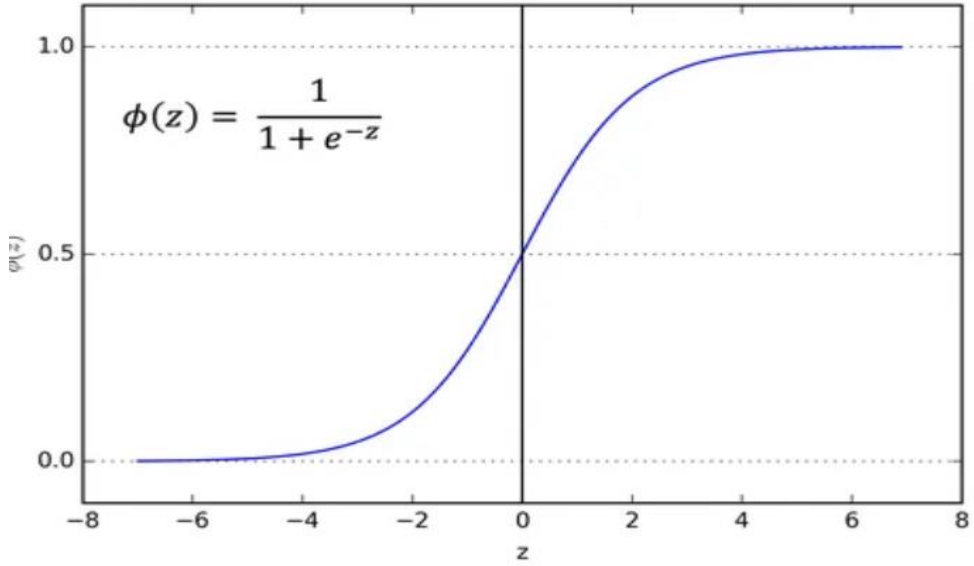
Linear Activation function

- The functional is linear. Therefore, no region will be employed to restrict the functions' output. The normal data input to neural networks is unaffected by the complexity or other factors.
- **Equation :** f(x) = x
- **Range :** (-infinity to infinity)
- It doesn’t help with the complexity or various parameters of usual data that is fed to the neural networks.

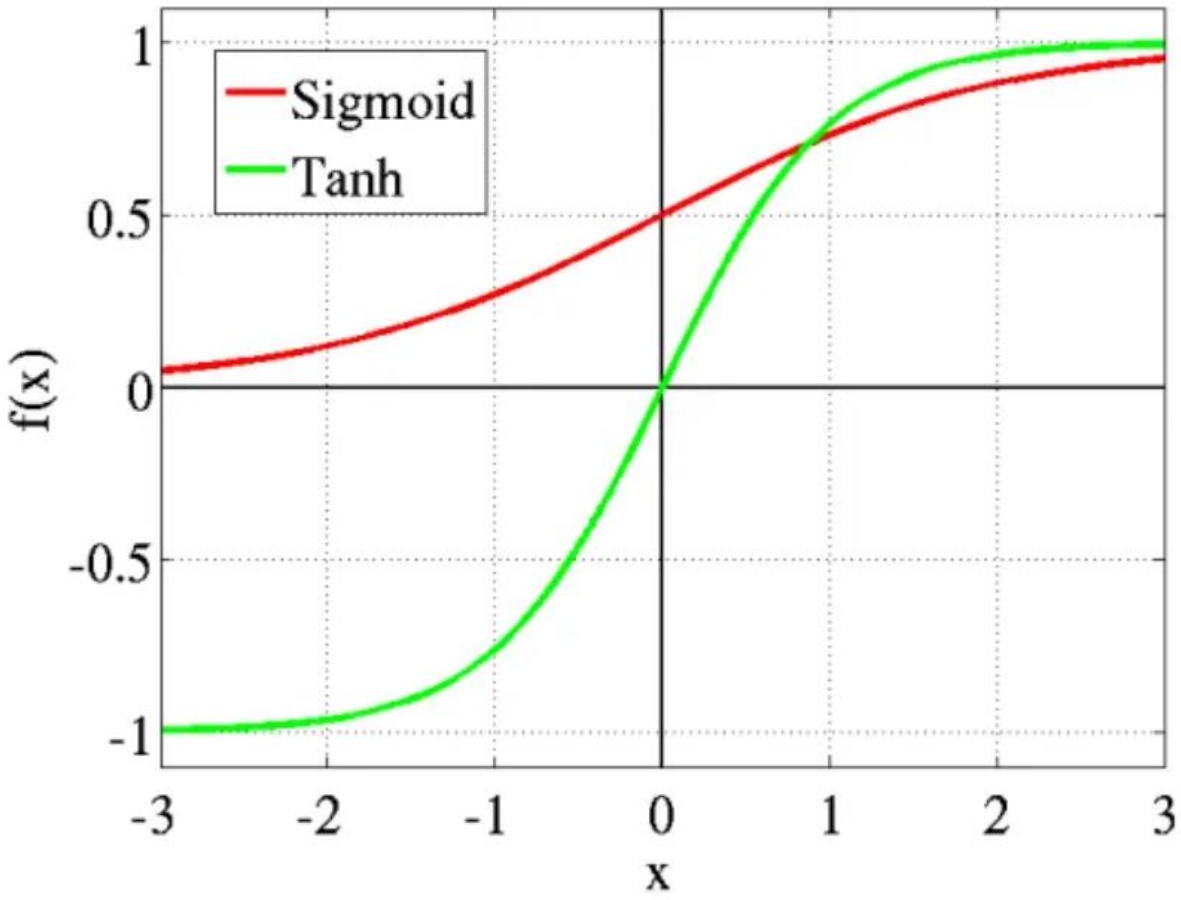


Non-linear Activation functions

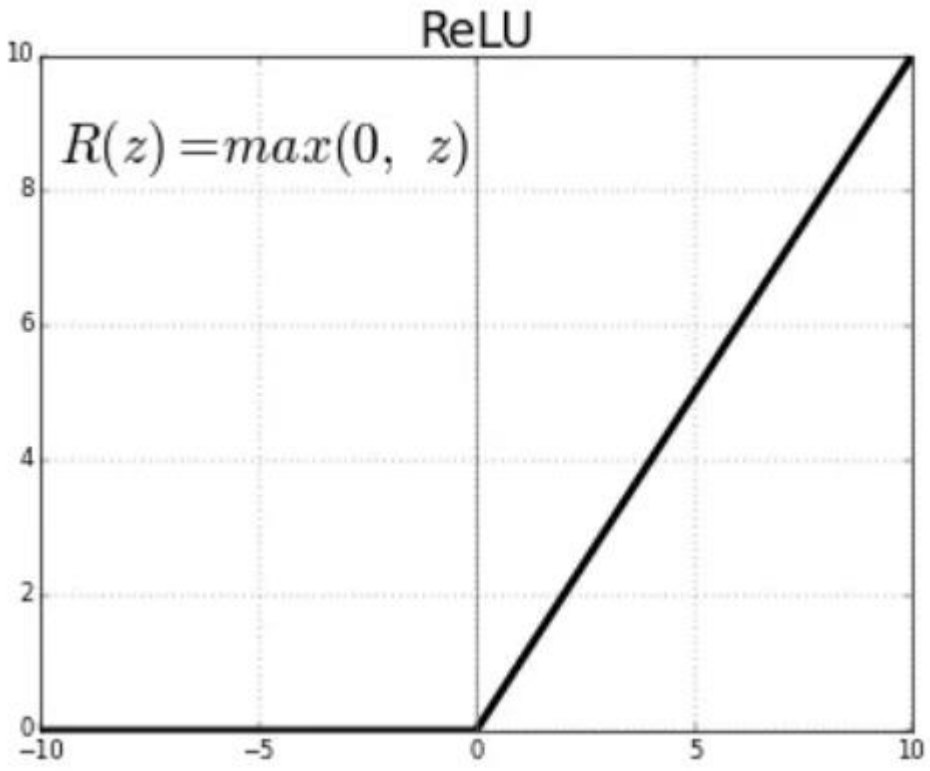
- The Nonlinear Activation Functions are the most used activation functions.
- **1. Sigmoid function -** The main reason why we use sigmoid function is because it exists between **(0 to 1).** Therefore, it is especially used for models where we have to **predict the probability** as an output.Since probability of anything exists only between the range of **0 and 1,** sigmoid is the right choice.



- **2. Tanh or hyperbolic tangent Activation Function -** tanh is also like logistic sigmoid but better. The range of the tanh function is from (-1 to 1). tanh is also sigmoidal (s - shaped).

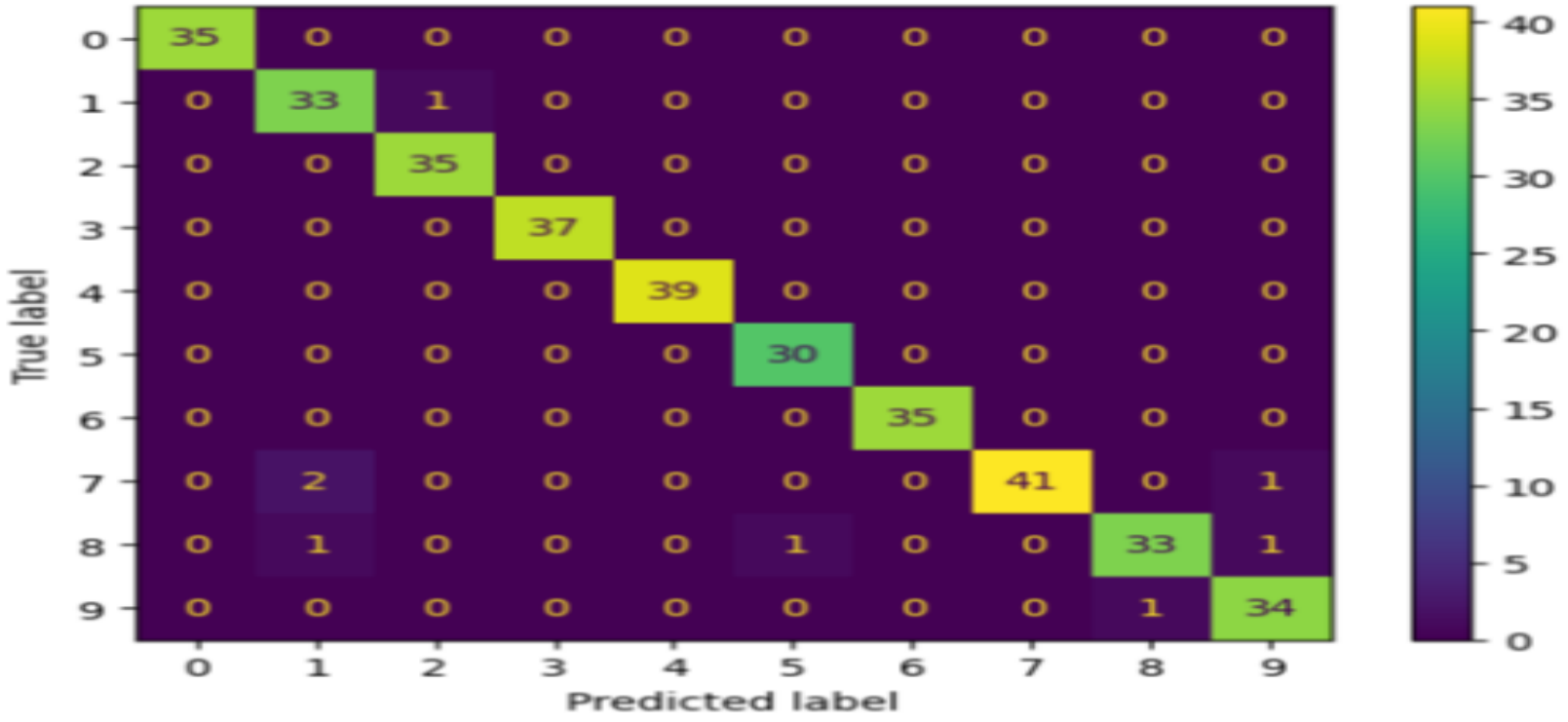


- **3. ReLU (Rectified Linear Unit) Activation Function -** Currently, the ReLU is the activation function that is employed the most globally. Since practically all convolutional neural networks and deep learning systems employ it. However, the problem is **that all negative values instantly become zero**, which reduces the model's capacity to effectively fit or learn from the data. This means that any negative input to a ReLU activation function immediately becomes zero in the graph, which has an impact on the final graph by improperly mapping the negative values



```
Step 1 # Importing the necessary libraries
import numpy as np
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import confusion_matrix,accuracy_score,ConfusionMatrixDisplay
import matplotlib.pyplot as plt
import pandas as pd
# Step 2 # Loading the dataset
df=load_digits()
print(df.target_names)
# Step 3 # Splitting the data into test and train # 80 - 20 Split
X_train, X_test, y_train, y_test = train_test_split(df.data,df.target, test_size=0.20, random_state=16)
# Step 4 # Making the Neural Network Classifier
nn=MLPClassifier()
# Step 5 # Training the model on the training data and labels
nn.fit(X_train,y_train)
# Step 6 # Testing the model i.e. predicting the labels of the test data.
y_pred=nn.predict(X_test)
# Step 7 # Evaluating the results of the model
acc=accuracy_score(y_test,y_pred)
con_mat=confusion_matrix(y_test,y_pred)
print("Accuracy is ",acc*100)
print(con_mat)
cm_display = ConfusionMatrixDisplay(confusion_matrix = con_mat)
cm_display.plot()
plt.show()
```

```
[0 1 2 3 4 5 6 7 8 9]
Accuracy is  97.77777777777777
[[35  0  0  0  0  0  0  0  0  0]
 [ 0 33  1  0  0  0  0  0  0  0]
 [ 0  0 35  0  0  0  0  0  0  0]
 [ 0  0  0 37  0  0  0  0  0  0]
 [ 0  0  0  0 39  0  0  0  0  0]
 [ 0  0  0  0  0 30  0  0  0  0]
 [ 0  0  0  0  0  0 35  0  0  0]
 [ 0  2  0  0  0  0  0 41  0  1]
 [ 0  1  0  0  0  1  0  0 33  1]
 [ 0  0  0  0  0  0  0  0  1 34]]
```



Loss Function

- A **loss function** is a function that **compares** the target and predicted output values; measures how well the neural network models the training data. When training, we aim to minimize this loss between the predicted and target outputs.
- he function we want to minimize or maximize is called the objective function or criterion. When we are minimizing it, we may also call it the cost function, loss function, or error function.
- The hyperparameters are adjusted to minimize the average loss — we find the weights, wT, and biases, b, that minimize the value of J (average loss).
- We can think of this akin to residuals, in statistics, which measure the distance of the actual y values from the regression line (predicted values) — the goal being to minimize the net distance.
- We cannot calculate the perfect weights for a neural network; there are too many unknowns. Instead, the problem of learning is cast as a search or optimization problem and an algorithm is used to navigate the space of possible sets of weights the model may use in order to make good or good enough predictions.
- In supervised learning, there are two main types of loss functions — these correlate to the 2 major types of neural networks: regression and classification loss functions

1. **Regression Loss Functions** — used in regression neural networks; given an input value, the model predicts a corresponding output value (rather than pre-selected labels); Ex. Mean Squared Error, Mean Absolute Error
2. **Classification Loss Functions** — used in classification neural networks; given an input, the neural network produces a vector of probabilities of the input belonging to various pre-set categories — can then select the category with the highest probability of belonging; Ex. Binary Cross Entropy, Categorical Cross Entropy;

Hyper parameters

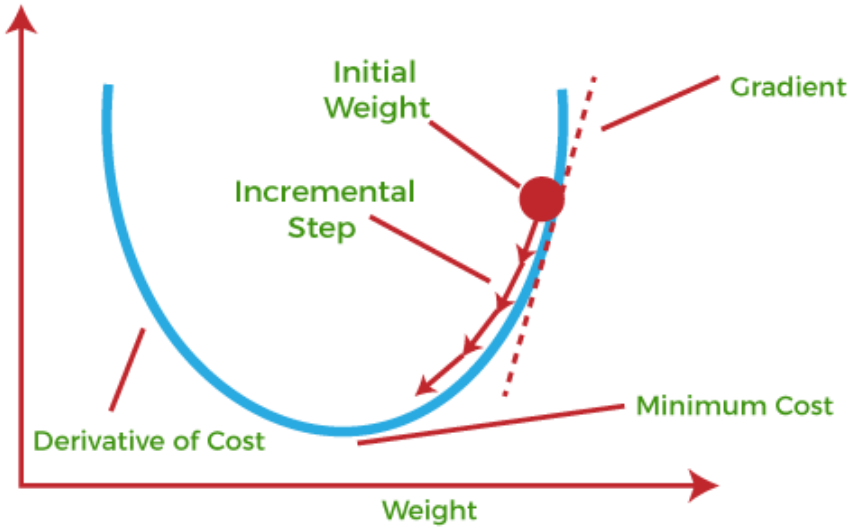
```
# Import MLPClassifer
from sklearn.neural_network import MLPClassifier
# Create model object
clf = MLPClassifier(hidden_layer_sizes=(6,5),
                    random_state=5,
                    verbose=True,
                    learning_rate_init=0.01)
# Fit data onto the model
clf.fit(X_train,y_train)
```

- Parameters:
- hidden_layer_sizes: it is a tuple where each element represents one layer and its value represents the number of neurons on each hidden layer.
- learning_rate_init: It used to controls the step-size in updating the weights.
- activation: Activation function for the hidden layer. Examples, identity, logistic, tanh, and relu. by default, relu is used as an activation function.
- random_state: It defines the random number for weights and bias initialization.
- verbose: It used to print progress messages to standard output.

Gradient Descent

- Typically, a neural network model is trained using the stochastic gradient descent optimization algorithm and weights are updated using the backpropagation of error algorithm.
- The "*gradient*" in gradient descent refers to an error gradient. The model with a given set of weights is used to make predictions and the error for those predictions is calculated.
- The gradient descent algorithm seeks to change the weights so that the next evaluation reduces the error, meaning the optimization algorithm is navigating down the gradient (or slope) of error.
- Gradient Descent is defined as one of the most commonly used iterative optimization algorithms of machine learning to train the machine learning and deep learning models. It helps in finding the local minimum of a function.

- If we move towards a negative gradient or away from the gradient of the function at the current point, it will give the **local minimum** of that function.
- Whenever we move towards a positive gradient or towards the gradient of the function at the current point, we will get the **local maximum** of that function.



- This entire procedure is known as Gradient Ascent, which is also known as steepest descent. ***The main objective of using a gradient descent algorithm is to minimize the cost function using iteration.*** To achieve this goal, it performs two steps iteratively:
- Calculates the first-order derivative of the function to compute the gradient or slope of that function.
- Move away from the direction of the gradient, which means slope increased from the current point by alpha times, where Alpha is defined as Learning Rate. It is a tuning parameter in the optimization process which helps to decide the length of the steps.

Backpropagation

- Backpropagation, or backward propagation of errors, is an algorithm that is designed to test for errors working back from output nodes to input nodes. It's an important mathematical tool for improving the accuracy of predictions in data mining and machine learning. Essentially, backpropagation is an algorithm used to quickly calculate derivatives in a neural network, which are the changes in output because of tuning and adjustments.
- Backpropagation Algorithm:
- Step 1: Inputs X, arrive through the preconnected path.
- Step 2: The input is modeled using true weights W. Weights are usually chosen randomly.
- Step 3: Calculate the output of each neuron from the input layer to the hidden layer to the output layer.
- Step 4: Calculate the error in the outputs
- Backpropagation Error= Actual Output – Desired Output
- Step 5: From the output layer, go back to the hidden layer to adjust the weights to reduce the error.
- Step 6: Repeat the process until the desired output is achieved.

- Features of Backpropagation:
- it is the gradient descent method as used in the case of simple perceptron network with the differentiable unit.
- it is different from other networks in respect to the process by which the weights are calculated during the learning period of the network.
- training is done in the three stages :
- the feed-forward of input training pattern
- the calculation and backpropagation of the error
- updation of the weight

Variants of Backpropagation

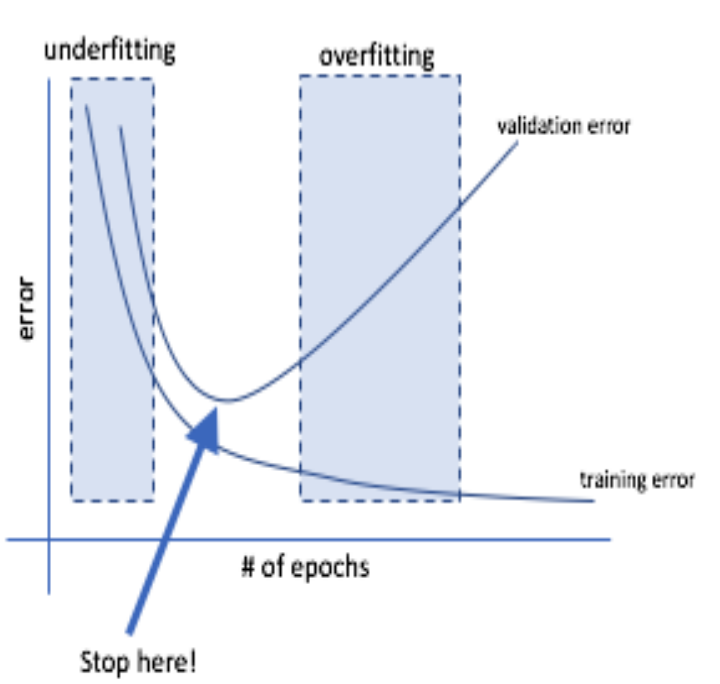
- There are two leading types of backpropagation networks:
- **Static backpropagation**. Static backpropagation is a network developed to map static inputs for static outputs. Static networks can solve static classification problems, such as optical character recognition (OCR).
- **Recurrent backpropagation**. The recurrent backpropagation network is used for fixed-point learning. This means that during neural network training, the weights are numerical values that determine how much nodes -- also referred to as neurons -- influence output values. They're adjusted so that the network can achieve stability by reaching a fixed value. involves computing the gradients of the loss function with respect to the network's parameters and then changing the parameters to minimize the loss. Recurrent neural networks have an internal feedback loop that enables data to be transferred from one phase to the next. Because the parameters of the network are shared across time steps, it is necessary to properly combine the gradients from each time step in order to update the parameters.

- **Underfit Model**. A model that fails to sufficiently learn the problem and performs poorly on a training dataset and does not perform well on a holdout sample.
- **Overfit Model**. A model that learns the training dataset too well, performing well on the training dataset but does not perform well on a hold out sample.
- **Good Fit Model**. A model that suitably learns the training dataset and generalizes well to the old out dataset.

- Overfitting of the model occurs when the model learns just ‘too-well’ on the train data. This would sound like an advantage but it is not. When a model is overtrained on training data, it performs worst on the test data or any new data provided.
- Technically, the model learns the details as well as the noise of the train data. This would hinder the performance of any new data provided to the model as the learned details and noise cannot be applied to the new data. This is the case when we say the performance of the model is not adequate.

Avoiding overfitting through Regularization

- There are several ways of avoiding the overfitting of the model such as **K-fold cross-validation, resampling, reducing the number of features**, etc.
- One of the ways is to apply **Regularization** to the model. Regularization is a better technique than Reducing the number of features to overcome the overfitting problem as in Regularization we do not discard the features of the model.
- Regularization is a technique that penalizes the coefficient.
- In an overfit model, the coefficients are generally inflated. Thus, Regularization adds penalties to the parameters and avoids them weigh heavily. The coefficients are added to the cost function of the linear equation.
- Thus, if the coefficient inflates, the cost function will increase. And Linear regression model will try to optimize the coefficient in order to minimize the cost function.
- Practically, you can check if the regression model is overfitting or not by RMSE. A good model has a similar RMSE for the train and test sets. If the difference is too large, we can say the model is overfitting to the training set.
- There are two kinds of techniques for adding penalities to the cost function. L1 Norm or LASSO term and L2 Norm or Ridge Term.



Why use neural networks?

- Ability to derive meaning from complicated or imprecise data.
- Extract patterns and detect trends that are too complex to be noticed by either humans or other computer techniques.
- Adaptive learning.
- Real Time Operation.
- Neural networks enable us to find solution where algorithmic methods are computationally intensive or do not exist.
- There is no need to program neural networks they learn with examples.
- Neural networks offer significant speed advantage over conventional techniques.

Applications of Neural Networks.

1. **Image Recognition and Computer Vision:** ANNs are extensively used in image classification, object detection, and segmentation tasks. Applications range from facial recognition in social media to medical image analysis and self-driving cars.
2. **Natural Language Processing (NLP):** ANNs power language modeling, machine translation, sentiment analysis, and chatbots. They help in understanding, generating, and processing human language, enabling applications like virtual assistants and language translation services.
3. **Healthcare:** ANNs aid in disease prediction, medical image analysis, drug discovery, and personalized medicine. They analyze medical data for diagnoses, assist in prognosis, and can even predict potential health risks.
4. **Finance and Stock Market Predictions:** ANNs are used for fraud detection, credit scoring, risk assessment, and stock market forecasting. They analyze financial data to make predictions and identify trends.
5. **Recommendation Systems:** ANNs play a vital role in recommendation engines for services like Netflix, Amazon, and Spotify. They analyze user behavior to provide personalized recommendations.
6. **Robotics and Control Systems:** ANNs contribute to robotics by enabling robots to learn and adapt to various tasks, such as object manipulation, navigation, and control in dynamic environments.
7. **Automated Driving:** ANNs are integral to autonomous vehicles, where they interpret sensor data, make decisions, and control the vehicle's actions in real-time.
8. **Predictive Maintenance:** ANNs assist in predictive maintenance by analyzing sensor data from machines and equipment, predicting potential failures, and optimizing maintenance schedules to prevent breakdowns.
9. **Gaming and Simulations:** ANNs are used in gaming for character behavior, opponent modeling, and generating realistic environments. They also contribute to simulations for training, testing, and modeling complex systems.
10. **Environmental Monitoring:** ANNs are employed in analyzing environmental data, such as weather forecasting, climate modeling, and pollution monitoring, aiding in understanding and managing environmental issues